

//Program on try with multiple catch blocks :

```
package com.ravi.basic;

public class MultipleCatch
{
    public static void main(String[] args)
    {
        try
        {
            int z = 10/0;
            IO.println("Z value is :"+z);

            int []arr = {100,200,300,400};
            IO.println(arr[4]);
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.err.println("Array is out of the bounds");
        }
        catch(ArithmeticException e)
        {
            System.err.println("Divide by zero problem");
        }
        catch(Exception e)
        {
            System.err.println("Some other problem");
        }

        IO.println("Main method completed!!!");
    }
}
```

```
package com.ravi.basic;

public class MultipleCatchBlocks {

    public static void main(String[] args)
    {
        IO.println("Main method started!!!");
        try
        {
            String str1 = "null";
            IO.println(str1.toUpperCase()); //NULL

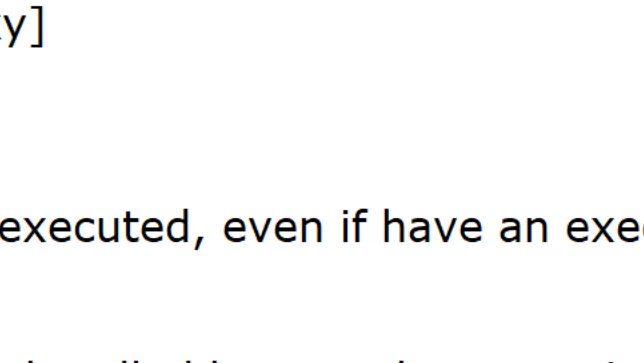
            String str2 = "Ravi";
            int x = Integer.parseInt(str2);
            IO.println("Number is :"+x);
        }

        catch(NumberFormatException | NullPointerException e)
        {
            if(e instanceof NumberFormatException)
            {
                System.err.println("Number is not in a proper format");
            }
            else if(e instanceof NullPointerException)
            {
                System.err.println("ref variable is pointing to null");
            }
        }

        System.out.println("Main method ended!!!");
    }
}
```

finally block : [100% Guaranteed for execution]

```
        try
        {
            database connection code;
            connection open;
            Making transaction of 18K;
            Updating the records;
            connection close;
            database close;
        }
        catch(Exception e)
        {
            IO.println("Exception Handled");
        }
    }
```



* We should never write any closing statements inside try block. [Very Risky]

* The main purpose of finally block to handle the resources.

* If our java control enters in the try block then finally block will be always executed, even if have an exception in the try block and we don't have catch block.

* If we use the combination of try and finally then only the resources will be handled but not the exception, on the other hand, If we use the combination of try-catch and finally then exception and resources both will be handled.

```
package com.ravi.basic;

public class FinallyBlockDemo
{
    public static void main(String[] args)
    {
        IO.println("Main method Started!!!");

        try
        {
            IO.println("Inside try block");
            IO.println(10/0);
        }
        finally
        {
            IO.println("Inside finally block");
        }

        IO.println("Main method Ended!!!");
    }
}
```

Note :- In the above program finally block will be executed, even we have an exception in the try block but here only the resources will be handled but not the exception.

```
package com.ravi.basic;

import java.util.InputMismatchException;
import java.util.Scanner;

public class FinallyWithTryCatch {

    public static void main(String[] args)
    {

        Scanner sc = null;
        try
        {

            sc = new Scanner(System.in);
            IO.print("Enter your Age :");
            int age = sc.nextInt();
            IO.println("Your Age is :"+age);
        }
        catch(InputMismatchException e)
        {
            IO.println("Catch Block");
            System.err.println("Input is not in a proper format");
        }
        finally
        {
            IO.println("Finally Block");
            sc.close();
        }

        IO.println("Main method completed!!!");
    }
}
```

Note : Here We have try-catch & finally so, exception & resources both will be handled